

CLOUDDRIVE

Cloud File Management Platform

Professional Engineering Documentation

System Design • Architecture • Implementation • Security • QA • Deployment

Mohammed Rizwan

B.Tech — Computer Science Engineering

github.com/riizwannx/cloud-drive

18 September 2026

Contents

1. Executive Overview
2. Project Identity, Background and Objectives
3. Functional Scope and User Workflows
4. System Architecture
5. Frontend Architecture
6. Backend Architecture and REST Lifecycle
7. Authentication and Session Lifecycle
8. Authorization, IDOR and BOLA Defense
9. File Upload Architecture and Security
10. File Download, Preview and Browser Delivery
11. Cloudinary Integration and PDF 401 Incident
12. Database Design and Data Model
13. Folder Hierarchy and Lifecycle Rules
14. Search and Query Design
15. Favorites and Sharing
16. Trash, Restore and Permanent Deletion
17. Dashboard, Quota and Resource Accounting
18. Input Validation and Injection Defense
19. API Resource Protection and Rate Limiting
20. HTTP Security, CORS, CSP and Cache Control
21. Error Handling, Secrets and Operational Security
22. Dependency and Supply-Chain Hardening
23. Deployment and Production Infrastructure
24. Production Verification and Release Evidence
25. QA Strategy and Test Matrix
26. Major Problems, Investigation and Solutions
27. Git, Change Control and Engineering Workflow
28. Performance, Reliability and Scalability
29. Known Limitations and V1 Boundaries
30. V2 Roadmap and Future Architecture
31. Portfolio, Viva and Interview Explanation
32. Engineering Lessons Learned
33. Final Project Status

Appendix A — Authoritative API Reference

Appendix B — Security Control Matrix

Appendix C — QA Matrix

Appendix D — Engineering Glossary

Abstract

CloudDrive is a production-deployed full-stack cloud file-management platform. It combines a React/Vite SPA, a Node.js/Express REST API, MongoDB Atlas and Cloudinary to provide authenticated file management with folders, favorites, sharing, Trash, recovery and secure private delivery.

This document presents the system as an engineering artifact. It explains the product requirements, architecture, component responsibilities, request lifecycles, data model, security controls, cloud-provider integration, debugging incidents, QA evidence, deployment topology, release history and V1 boundaries. The document intentionally distinguishes verified implementation from future work.

1. Executive Overview

CloudDrive is a production-deployed cloud file-management application that combines a React/Vite single-page interface, a Node.js/Express REST API, MongoDB Atlas for structured metadata and Cloudinary for binary assets. The system is designed around a personal-drive workflow: authenticate, manage files, organize them in nested folders, search, favorite, share, recover deleted files and permanently delete files.

The project is documented as an engineering system rather than only as a feature list. The important design decisions are the separation of metadata from binary storage, server-side authorization, cryptographic session handling, file-content validation, atomic quota accounting, protected cloud delivery, rate limiting, security headers and explicit lifecycle rules.

This final edition also records boundaries honestly. V1 is a single-file upload system with PDF/JPEG/JPG/PNG support and a 5 MB request limit. Move File, multi-file upload, drag-and-drop, resumable uploads, dashboard category charts, category filter pills, column sorting, bulk Trash purge and folder-level collaboration are not implemented.

2. Project Identity, Background and Objectives

Project name: CloudDrive. Author: Mohammed Rizwan. Repository: <https://github.com/riizwannx/cloud-drive>. CloudDrive was developed as a full-stack engineering project to turn the familiar cloud-drive concept into a real distributed web application with authentication, authorization, storage, organization, sharing and production deployment.

The core problem is not simply storing bytes. A useful cloud-drive system must maintain relationships between users, file metadata, folders, favorites, sharing state, deletion state and quota while keeping private resources isolated between accounts. The binary payload should also be handled by a suitable object-storage provider rather than becoming a database responsibility.

Primary objectives were to build a usable V1, protect private resources, validate uploads at the server boundary, maintain reliable storage accounting, integrate private Cloudinary delivery, provide recoverable deletion, expose a clear REST API and verify the deployed application rather than relying only on local screenshots.

3. Functional Scope and User Workflows

The authenticated workflow starts at registration or login. After authentication, the dashboard provides storage, file, folder and shared-link metrics plus recent files and Quick Actions. My Files is the main operational surface. Users can upload a supported file, preview it, download it, rename it, favorite it, share it or move it to Trash.

Folder workflows support nested hierarchies through a parentFolder relationship. A folder can be created, renamed, inspected and deleted subject to lifecycle rules. Files may be associated with a folder during upload, but V1 does not contain an interactive Move File operation for relocating an existing file.

Sharing creates a public guest-access path for a selected active file. The owner controls sharing state, while public access is separated from the authenticated workspace. Moving a shared file to Trash revokes the public link. Trash is a soft-delete state; permanent deletion is deliberately restricted to files already in Trash.

4. System Architecture

The production architecture is intentionally decoupled. Vercel serves the React/Vite SPA, Render runs the Express API, MongoDB Atlas stores structured application state, and Cloudinary stores binary assets. The browser communicates with the API over HTTPS. The API is the main trust boundary and is responsible for authentication, authorization, validation, resource controls and provider access.

MongoDB stores users, password hashes/security state, file records, folder relationships, favorites, sharing state, Trash state, timestamps and quota information. Cloudinary stores file bytes. This separation lets the application authorize a file using MongoDB metadata before generating or streaming a private asset delivery path.

The normal request path is: UI → Axios service → HTTPS → Express middleware → route/controller/service → MongoDB and/or Cloudinary → sanitized HTTP response → UI state update. Cross-cutting middleware applies JSON limits, CORS, Helmet, rate limits and error handling before business logic.

CloudDrive — Production Architecture

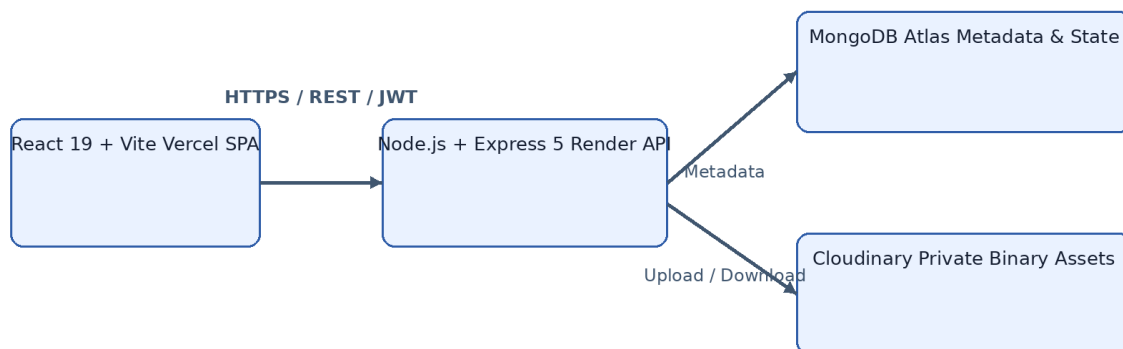


Figure 1. Production architecture and trust boundaries.

5. Frontend Architecture

The frontend is a React 19 SPA built with Vite 8 and Tailwind CSS 4, with React Router DOM 7.18.3 for navigation. The application shell contains MainLayout, AppSidebar and Navbar. Page-level surfaces include Dashboard, My Files, Favorites, Shared, Trash, Folders, Folder Details and Settings.

My Files separates the toolbar, table, rows and upload control. File rows currently expose favorite, preview, download, share, rename and delete actions. The toolbar search filters loaded filenames client-side. This is distinct from the backend search capability and should not be described as a single unified search engine.

The frontend stores the JWT in localStorage for the current V1 session model. Axios attaches the token to authenticated requests and has a 401 interceptor that clears the token and redirects protected routes to login. Logout attempts the backend invalidation endpoint and then clears client state even if the request fails.

6. Backend Architecture and REST Lifecycle

The Express 5 backend is organized around routes, controllers/services and Mongoose models. `app.js` configures JSON body limits, CORS, Helmet, rate limiters, route mounting and the global error handler. `server.js` starts the application and the database configuration establishes the MongoDB connection.

An authenticated request carries an `Authorization: Bearer` token header. The authentication middleware verifies the JWT using the configured secret and HS256 algorithm, validates the user identifier, checks `tokenVersion` against the database and attaches the authenticated identity to the request. Private responses also receive anti-cache directives.

Controllers perform input checks before database operations. ObjectIds are validated, string fields are bounded, search expressions are escaped and filenames/folder names are sanitized. Authorization is then applied separately: valid input is not automatically authorized input.

7. Authentication and Session Lifecycle

Registration validates username, email and password, then stores the password only as a bcrypt hash. The User model keeps the password field excluded by default. Login explicitly selects the password hash for comparison and returns a generic invalid-credentials response on failure.

JWTs use a minimal payload containing the user identifier and `tokenVersion`, with HS256 pinned during verification and a seven-day expiration. The `tokenVersion` mechanism gives the server a revocation control despite JWTs being otherwise stateless: logout and password change increment the stored version, making older tokens invalid.

Login is protected by two failed-attempt dimensions: an IP-based limiter and an account/email-based limiter. Password change has a dedicated user-keyed limiter. This limits credential abuse without replacing authentication or ownership checks.

Authentication & Session Lifecycle

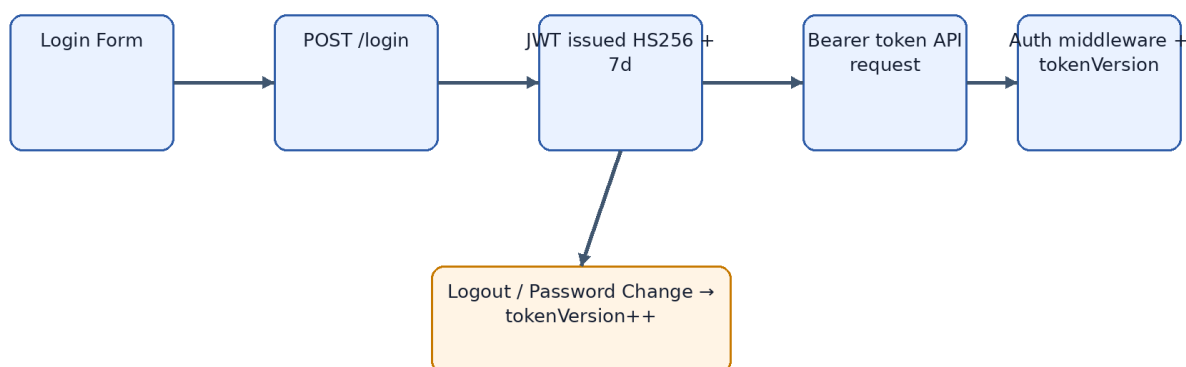


Figure 2. Authentication and token-revocation lifecycle.

8. Authorization, IDOR and BOLA Defense

Every private file, folder and user-scoped operation is treated as tenant-scoped data. A client-controlled resource ID is never sufficient by itself. The server validates the identifier and checks that the requested record belongs to the authenticated user before allowing the operation.

This prevents insecure direct object reference and broken object-level authorization patterns such as changing another user's favorite flag, downloading another user's file, deleting another user's folder or accessing another user's metadata by guessing an ID.

Authorization is layered after input validation. An attacker can submit a syntactically valid `ObjectId` that still belongs to another account. The security boundary is therefore the combination of validated identifiers, authenticated identity and ownership queries.

9. File Upload Architecture and Security

V1 accepts one file per request through a standard browser picker. The current accepted content types are PDF, JPEG/JPG and PNG, with a 5 MB maximum. The backend uses bounded multipart handling and validates the uploaded buffer before treating it as a legitimate file.

Validation checks the declared metadata and file signature rather than trusting the browser-provided MIME type alone. Filename sanitization removes dangerous control/traversal characters. The file is then sent to Cloudinary using `upload_stream` after an atomic quota reservation.

Quota reservation uses a database-level conditional update. This prevents two simultaneous uploads from both reading the same remaining capacity and exceeding the account limit. If a later database or provider step fails, the reserved quota is refunded and the cloud asset is cleaned up when applicable.

Secure File Upload Flow

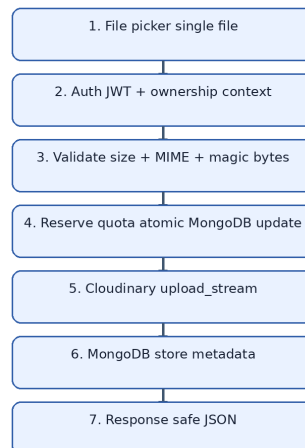


Figure 3. Secure upload sequence.

10. File Download, Preview and Browser Delivery

Private file delivery begins with an authenticated API request and an ownership check. The backend obtains the corresponding Cloudinary asset and generates a signed, time-limited private delivery URL rather than exposing a permanently public asset URL.

Download responses use attachment semantics where appropriate and include nosniff protection. Preview uses the same authorized binary retrieval path and creates a browser blob URL. Chromium's

native PDF viewer is outside the React DOM, so automation cannot inspect the viewer itself; the functional evidence is the successful authorized PDF response and browser opening behavior.

This design keeps the browser from bypassing application authorization by directly discovering a public Cloudinary resource. The application remains responsible for deciding whether the current user can access the file.

11. Cloudinary Integration and PDF 401 Incident

Cloudinary is responsible for binary asset storage and delivery, while MongoDB remains responsible for application metadata and authorization state. The integration uses the Cloudinary Node SDK and streaming upload for bounded file handling.

A significant debugging incident occurred when PDF delivery returned HTTP 401 even though upload had succeeded. The investigation separated upload from delivery and identified that private/raw PDF delivery required the correct resource-type handling and signed private download mechanism.

The solution centralized resource-type-aware private delivery and signed URL generation instead of weakening authentication or making the PDF public. Subsequent production download and preview checks returned the expected PDF response.

12. Database Design and Data Model

The User document stores identity, password hash/security state, tokenVersion and storage accounting. The File document stores ownership, original name, size/type, Cloudinary identifiers, folder association, favorite state, sharing state/token, Trash state and lifecycle timestamps. Folder documents represent nested hierarchy through parentFolder and ownership.

Database state is authoritative for ownership and lifecycle decisions. Quota is represented as a logical byte counter even though physical bytes live in Cloudinary. File state transitions such as Trash and sharing revocation update MongoDB before the UI is refreshed.

Indexes and bounded queries support common user-scoped operations. The application avoids treating MongoDB as an unbounded query interpreter: input types are constrained and query operators cannot be smuggled through ordinary string fields.

13. Folder Hierarchy and Lifecycle Rules

Folders support nested structures through a parentFolder reference. Creating a subfolder establishes the parent relationship, and the frontend can navigate into a folder and show breadcrumb context.

Folder deletion is guarded by lifecycle rules. Active files or active subfolders prevent unsafe removal. Trashed files can be disassociated when their containing folder is removed, preventing a restored file from referencing a container that no longer exists.

These rules make folder state more than a visual tree. The database relationship must remain valid through deletion, restoration and navigation, so the backend enforces the lifecycle rather than trusting the UI.

14. Search and Query Design

The My Files toolbar performs immediate client-side filename filtering over files already loaded into the page. This makes the interaction responsive and avoids a request on every keystroke. The backend also exposes a search capability for server-side retrieval.

Search input is bounded and regex metacharacters are escaped before being used in a regular-expression query. This reduces regex injection and pathological pattern behavior. Pagination also uses explicit limits to avoid unbounded result sets.

The current Navbar search field is not equivalent to the My Files toolbar filter. The final documentation therefore records the distinction instead of presenting an unsupported global-search experience.

15. Favorites and Sharing

Favorites are represented by an `isFavorite` state on the file record. The server checks ownership before toggling the state, and trashed files cannot be favorited. The dedicated Favorites view is therefore a filtered representation of the same authorized file records.

Sharing changes the access model and is consequently more security-sensitive than a personal favorite. A share token is generated with cryptographically secure randomness and represented as a hexadecimal value. The public route validates the token and sharing state without requiring a JWT.

Public metadata is deliberately limited to safe file information. Owner identifiers and Cloudinary internals are not exposed. Revoking a share clears its active sharing state, and Trash automatically revokes sharing for the affected file.

16. Trash, Restore and Permanent Deletion

Trash is implemented as soft deletion. Normal deletion sets `isTrashed` and records the deletion time while retaining the Cloudinary asset and quota usage so the user can restore the file.

Restore returns the file to active state. If its former folder still exists, the relationship can be preserved; if the folder was removed while the file was in Trash, restoration moves the file to root rather than creating an invalid relationship.

Permanent deletion is allowed only from Trash. The backend atomically claims the file before contacting Cloudinary, preventing concurrent deletion requests from both processing the same record. After successful cloud deletion, quota is reduced and the MongoDB record is removed. If cloud deletion fails, the claim is rolled back so the file remains recoverable.

Trash & Permanent Deletion Lifecycle

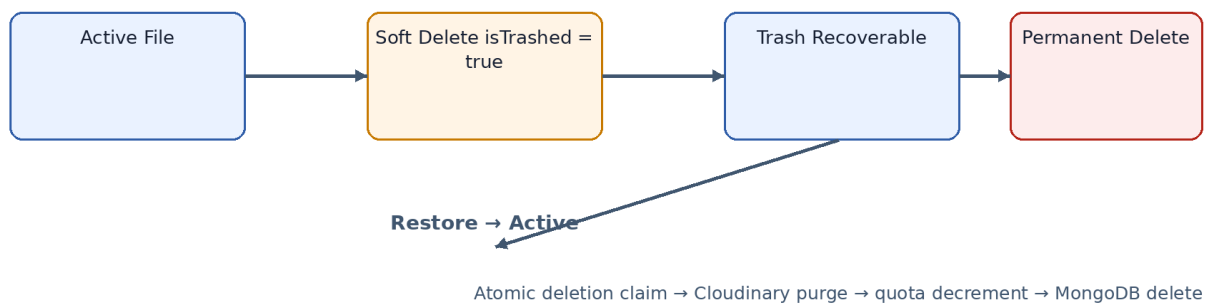


Figure 4. File deletion lifecycle and concurrency control.

17. Dashboard, Quota and Resource Accounting

The dashboard reports storage used, storage limit, file count, folder count, shared-link count and recent files. The current account limit is 5 GB and storageUsed is maintained as a byte counter.

The important design point is atomicity. A naive read-check-write sequence can race when two uploads arrive together. The implementation combines the capacity condition and increment in a MongoDB findOneAndUpdate operation so only requests that satisfy the quota can reserve space.

Quota is refunded when an upload pipeline fails after reservation. Permanent deletion decrements the counter after successful physical deletion. This creates a consistent logical accounting model across the database and Cloudinary.

18. Input Validation and Injection Defense

User input crosses several interpreters: JavaScript, MongoDB, regular expressions, HTTP headers and cloud-provider APIs. Validation therefore happens at the API boundary and is not delegated to the React interface.

NoSQL/operator injection is reduced by enforcing expected primitive types and rejecting nested operator-shaped input where ordinary strings are expected. ObjectIds are validated before queries, search patterns are escaped, and filenames/folder names are sanitized and length-limited.

Validation and authorization remain separate. Input can be valid but unauthorized. This separation is important for IDOR/BOLA defense and for keeping business rules explicit.

19. API Resource Protection and Rate Limiting

The API limits JSON bodies to 50 KB, uploads to 5 MB and one file per upload request. Pagination, search inputs and share tokens are bounded. These limits reduce accidental and malicious resource consumption.

Dedicated rate limiters protect registration, login, password change, upload, download, search, sharing, dashboard operations and permanent deletion. Login has separate IP and account

dimensions so one abusive client cannot simply rotate between failure keys without consequence.

The application sets trust proxy appropriately for its Render deployment so IP-based rate controls can use the client address as intended. Rate limiting controls volume; it does not decide ownership or authorization.

20. HTTP Security, CORS, CSP and Cache Control

Helmet supplies standard security headers on the Express API. The frontend adds Vercel response headers including a restrictive Content Security Policy, X-Frame-Options DENY, nosniff, Referrer-Policy and a restrictive Permissions-Policy.

CORS is restricted to the production Vercel origin and permitted local development origin. CORS is treated as a browser policy rather than a replacement for backend authorization.

Authenticated/private API responses use private, no-cache, no-store, must-revalidate together with Pragma and Expires directives. Live production verification confirmed these headers rather than relying only on source configuration.

21. Error Handling, Secrets and Operational Security

Expected validation and authentication failures are returned as controlled client responses.

Unexpected failures are logged server-side and represented to the client by generic error information, avoiding stack traces and infrastructure details.

JWT secrets and Cloudinary credentials are server-side environment variables. The frontend only receives the public API URL required for its deployment. Database connection error logging sanitizes credentials before writing diagnostic information.

Production logs were reviewed for development-only debug leakage, and the release audit included a secret scan. Reticle was used only as a local QA layer and its development artifacts were removed before release.

22. Dependency and Supply-Chain Hardening

The application uses lockfiles for repeatable dependency installation. The final audited stack includes React 19.2.7, Vite 8.1.1, Express 5.2.1, Mongoose 9.7.4, Cloudinary 2.11.0, Multer 2.3.0, jsonwebtoken 9.0.3, bcrypt 6.0.0, Helmet 8.3.0, express-rate-limit 8.7.0, Axios 1.18.1 and cors 2.8.6.

Multer uses bounded memory storage because the upload buffer is inspected before being streamed to Cloudinary. This makes the file-size ceiling especially important because memory-backed requests must never be allowed to grow without a strict limit.

Future hardening can include automated vulnerability scanning, dependency update policy and build provenance. These are recorded as improvements rather than falsely presented as existing CI capabilities.

23. Deployment and Production Infrastructure

The active V1 production topology is Vercel + Render + MongoDB Atlas + Cloudinary. Vercel serves the built React SPA, Render runs the Express API, Atlas manages the database and Cloudinary manages binary assets.

The frontend uses `VITE_API_URL` to target the Render /api service. The backend allows the production Vercel origin through CORS. HTTPS protects browser-to-service traffic in production.

AWS EC2/S3, Docker/Compose, Nginx, Prometheus and Grafana are not active V1 production infrastructure. They appeared in earlier planning or unrelated development discussions and are therefore not represented as deployed components.

24. Production Verification and Release Evidence

Production verification included live HTTP checks for the frontend and backend, security-header inspection, authenticated cache-control checks, CORS behavior, API URL verification and browser smoke tests.

Functional production behavior verified included login, file upload, PDF preview, PDF download, file sharing, unauthenticated public-share access and logout. The deployed frontend and backend were reachable at their configured Vercel and Render addresses during verification.

The final QA cleanup release is commit 0291138 with message 'fix: finalize CloudDrive dashboard actions and QA cleanup'. The security milestone was commit 47fe228 with message 'security: complete CloudDrive security hardening'. At the documentation audit point, README.md remained an intentional unstaged documentation change.

25. QA Strategy and Test Matrix

QA combined source inspection, API behavior, browser interaction and deployment verification. The goal was to validate both visible workflows and invisible contracts such as ownership, token lifecycle, cloud delivery, quota consistency, security headers and rate limiting.

Recorded QA areas covered login/session behavior, protected routes, dashboard Quick Actions, My Files, real upload, file actions, PDF preview, download, folder creation, favorites, sharing, Trash, search, settings/password validation, dashboard consistency, security regression and final release audit.

Two important limitations were preserved. PDF preview could not be inspected inside the native browser PDF viewer by Reticle, although the authorized response and runtime behavior passed. Trash lifecycle test #13 remains incomplete/skipped in the Reticle record because native window.confirm could not be controlled and the correct QA account/file state was not consistently available.

26. Major Problems, Investigation and Solutions

Cloudinary PDF 401: Upload succeeded but private PDF delivery returned 401. The investigation separated storage from delivery and corrected the private/raw resource delivery path with signed URLs.

Quick Actions routing: Dashboard cards initially behaved like static controls. Source inspection identified the missing useNavigate wiring. The four actions were connected to /files, /folders, /shared

and /trash and then re-tested successfully.

Reticle release artifacts: Local QA introduced configuration/helper files into the working tree. The final audit detected them, removed the artifacts, updated ignore rules and reverted unrelated package/Vite changes. This prevented test tooling from leaking into production.

27. Git, Change Control and Engineering Workflow

The engineering workflow separated implementation from release control. Antigravity was used as the implementation/QA executor, ChatGPT as architecture/review, and the user as project owner and final QA authority. Git commits and pushes were intentionally grouped rather than created after every individual fix.

The security hardening milestone consolidated major controls including injection protection, brute-force controls, authorization, upload security, JWT lifecycle, resource limits, headers, CORS, cache behavior, error handling and deletion consistency.

The final QA cleanup retained only intended application changes and removed local testing artifacts. This illustrates why a final diff/status audit is necessary even when individual feature tests have passed.

28. Performance, Reliability and Scalability

V1 favors correctness and manageable operational complexity over premature distributed infrastructure. Database metadata and cloud binaries are separated, queries are bounded, uploads are capped, rate limits are applied and quota operations are atomic.

The main scaling pressure points are upload throughput, concurrent downloads, database query volume, search over large file collections, Cloudinary bandwidth and synchronous deletion workflows. A larger deployment could introduce background jobs, queues, resumable uploads, richer indexes and observability.

Reliability is strengthened by compensating actions in the upload and deletion pipelines. Quota reservations can be refunded, cloud deletion failure can roll back the deletion claim, and lifecycle state prevents invalid restoration relationships.

29. Known Limitations and V1 Boundaries

The current upload UI is single-file and picker-based. There is no drag-and-drop, multi-file batch upload or resumable/chunked upload. The maximum request size is 5 MB and accepted formats are PDF/JPEG/JPG/PNG.

There is no Move File operation for an existing file, no folder-level sharing/collaboration, no bulk empty-Trash purge, no category distribution dashboard, no category filter pills and no column sorting. These are documented as product boundaries, not defects in the current implementation.

The root LICENSE file is currently unpopulated; backend package metadata uses ISC while the frontend package is private. This is a documentation/repository hygiene point to address if formal redistribution is required.

30. V2 Roadmap and Future Architecture

A natural V2 begins with higher-throughput file operations: drag-and-drop, multi-file uploads, Move File, bulk Trash operations and resumable/chunked uploads. These should preserve the existing authorization and quota invariants rather than bypassing them.

Collaboration can extend file-level sharing into folder-level permissions, shared folders and role-based access. Version history can add immutable file revisions and recovery points. Search can evolve toward server-side indexed search with richer filters and sorting.

At larger scale, asynchronous workers and queues can move expensive provider operations out of synchronous request paths. Observability can add metrics, structured logs and tracing. Any such evolution should retain the same core trust model: backend authorization before asset access and explicit lifecycle state transitions.

31. Portfolio, Viva and Interview Explanation

A concise explanation is: 'CloudDrive is a production-deployed full-stack cloud file-management platform. React/Vite provides the SPA, Express provides the REST API, MongoDB Atlas stores metadata and Cloudinary stores private binary assets. I implemented authentication, authorization, secure uploads, quota accounting, folders, favorites, sharing, Trash and production security controls.'

The strongest engineering discussion points are not visual features alone. Explain why authorization is server-side, why MIME type cannot be trusted, why magic-byte validation is used, why quota reservation is atomic, why JWT tokenVersion exists, why private Cloudinary delivery uses signed URLs and why deletion is split into soft-delete and permanent deletion.

For a viva, also explain the limitations honestly. Being able to say what is not implemented and why is stronger engineering communication than claiming unsupported functionality.

32. Engineering Lessons Learned

The project demonstrated that production engineering is largely about interactions between components. A correct React button is not enough if the API lacks authorization; a successful upload is not enough if private delivery returns 401; and a correct quota formula is not enough if concurrent requests can race.

Source inspection, runtime evidence and provider behavior each answered different questions. Browser automation found UI gaps, API tests verified HTTP contracts, production checks verified deployment configuration, and release audits caught local tooling artifacts that feature tests did not reveal.

The central lesson is to design trust boundaries explicitly. The browser is untrusted, MongoDB is the source of application state, Cloudinary is the binary provider, and the API decides whether an operation is allowed.

33. Final Project Status

CloudDrive V1 is production-deployed and has undergone security hardening, functional QA and release cleanup. The major security milestone is complete, the final QA cleanup is released as 0291138, and the active production topology is documented accurately.

The final state should be understood as a deliberate V1, not as an unfinished prototype. The implemented scope is stable enough to document, while the limitations define clear next engineering steps.

This document is intended to serve simultaneously as a technical record, maintenance reference, viva preparation document and portfolio evidence of the engineering process used to build and harden CloudDrive.

Appendix A — Authoritative API Reference

The following inventory reflects the documented V1 route surface. Health is included as an operational endpoint.

Method	Endpoint	Purpose
	/api/users/register	Create account
POST	/api/users/login	Authenticate
	/api/users/logout	Invalidate current token version
GET	/api/users/profile	Get authenticated profile
	/api/users/change-password	Change password and revoke older sessions
POST	/api/files/upload	Upload one validated file
	/api/files	List user's files
GET	/api/files/folder/:folderId	List files in a folder
	/api/files/download/:id	Authorized private download
PUT/PATCH	/api/files/:id	Update file metadata/state
	/api/files/:id	Move file to Trash
DELETE	/api/files/permanent/:id	Permanently delete a trashed file
	/api/files/restore/:id	Restore from Trash
PATCH	/api/files/favorite/:id	Toggle favorite
	/api/files/favorites	List favorites
GET	/api/files/trash	List Trash
	/api/files/search	Search files
PATCH	/api/files/share/:id	Create/enable file sharing
	/api/files/share/remove/:id	Revoke sharing
GET	/api/files/shared	List shared files
	/api/files/shared/:token	Public shared file access
GET	/api/files/shared/:token/info	Public safe metadata
	/api/folders	Create folder
GET	/api/folders	List folders
	/api/folders/:id	Get folder details
PATCH	/api/folders/:id	Rename/update folder
	/api/folders/:id	Delete folder under lifecycle rules
GET	/api/dashboard	Dashboard metrics
	/api/health	Health check

Appendix B — Security Control Matrix

Control	Implementation	Purpose
Session revocation	HS256 pinned, 7-day expiry, minimal payload	Prevent weak/ambiguous token verification
	tokenVersion	Invalidate old JWTs on logout/password change
Login protection	bcrypt hashes; password excluded by default	Avoid plaintext credential storage
	IP + account failed-attempt limits	Reduce brute-force abuse
Quota	5 MB; PDF/JPEG/JPG/PNG; magic bytes	Reject unsupported or disguised content
	Atomic reservation/refund	Prevent concurrent over-allocation
Public sharing	Owner checks on private resources	Prevent IDOR/BOLA
	Cryptographically random token	Avoid guessable share URLs
CORS	Helmet + CSP + nosniff + frame controls	Reduce browser attack surface
	Production origin allowlist	Restrict browser cross-origin access
Rate limits	no-cache/no-store/must-revalidate	Prevent reuse of private responses
	Endpoint-specific limits	Control resource abuse
	Generic client errors	Avoid leaking internal details

Appendix C — QA Matrix

Area	Result	Evidence / limitation
	PASS	Authenticated browser flow and API behavior
Logout + protected route	PASS*	Frontend cleanup/protected redirect passed; one run had local logout API unavailable
	PASS	All four routes verified
My Files	PASS	List/actions rendered and API responses succeeded
	PASS	PDF upload returned 201 and file appeared
PDF preview	PASS*	Functional; native PDF viewer outside DOM automation
	PASS	PDF response/download behavior verified
Folder creation	PASS	Folder created and ownership verified
	NOT IMPLEMENTED	No UI or backend move route
Favorites	PASS	Favorite/unfavorite state verified
	PASS	Private owner flow + public guest access verified
Trash lifecycle	INCOMPLETE	Native confirm + account mismatch blocked full Reticle run
	PASS	Client-side filename filtering verified
Settings/password validation	PASS	Invalid inputs blocked without network submission
	PASS	Metrics and recent files stable after refresh
Security regression	PASS	Auth, ownership, injection, headers, limits reviewed
	PASS	Artifacts cleaned, build/syntax/diff checks passed

* Indicates a documented qualification/limitation, not an unqualified PASS.

Appendix D — Engineering Glossary

API: Application Programming Interface; CloudDrive's REST interface between frontend and backend.

BOLA: Broken Object Level Authorization; an access-control failure where a caller can access another user's object.

CORS: Cross-Origin Resource Sharing; browser policy controlling permitted cross-origin requests.

CSP: Content Security Policy; browser control restricting permitted resource sources.

IDOR: Insecure Direct Object Reference; a common form of object authorization weakness.

JWT: JSON Web Token used for authenticated API sessions in V1.

MIME type: Content-type metadata describing the intended media format of a file.

Magic bytes: File-signature bytes used to validate actual content rather than trusting only the filename or MIME header.

Soft delete: State transition into Trash without immediately destroying the underlying asset.

Quota reservation: Atomic database operation that reserves storage capacity before upload completion.

TokenVersion: Database-backed session generation used to invalidate previously issued JWTs.

SPA: Single-page application; the React frontend served as one client application.